Presentado por María Victoria Huesca Peláez

Caso Práctico - Módulo 1
NovaBank Digital Services

Documentación del proyecto

Índice

1. Introducción..... 3
2. Descripción funcional..... 4
    2.1 Menú principal..... 4
    2.2 Gestión de clientes..... 4
    2.3 Gestión de cuentas..... 6
    2.4 Operaciones financieras..... 7
    2.5 Consultas..... 8
3. Modelo de datos..... 9
    Relaciones:..... 9
    Diagrama entidad-relación:..... 10
4. Decisiones de implementación..... 10
5. Scripts SQL..... 11
6. Testing..... 12
    Clases de test..... 13
    Ejecución de los tests..... 13
7. Control de versiones con Git..... 14
    Historial de commits..... 14
    Estructura de ramas..... 15

1. Introducción.

NovaBank Digital Services es un sistema de gestión bancaria desarrollado como proyecto práctico del Módulo 1 del Plan Formativo Backend Java de NTT Data.

El objetivo de este módulo es construir una versión inicial del sistema centrada en los fundamentos del lenguaje Java y la Programación Orientada a Objetos. Para mantener el foco en la lógica del dominio, la aplicación funciona completamente en consola, sin interfaces gráficas ni servicios web.

El sistema permite a los empleados del banco realizar las siguientes operaciones:

- Registro y búsqueda de clientes.
- Creación y consulta de cuentas bancarias.
- Ejecución de operaciones financieras: depósitos, retiradas y transferencias entre cuentas.
- Consulta de saldos e historial de movimientos, con filtrado por rango de fechas.

Los datos se almacenan en memoria mediante estructuras estándar de Java (HashMap). Al cerrar la aplicación todos los datos se pierden, lo cual es completamente intencionado en este módulo.

2. Descripción funcional

La aplicación es interactiva en consola. Al iniciar se muestra un menú principal y el usuario navega por los submenús seleccionando opciones numéricas.

2.1 Menú principal

```
=====================================
 NOVABANK - SYSTEM OPERATIONS
=====================================
1. Client management
2. Account management
3. Financial operations
4. Inquiries
5. Exit
Select an option:
```

2.2 Gestión de clientes

Crear cliente

El sistema solicita nombre, apellidos, DNI/NIF, email y teléfono. Valida que todos los campos sean obligatorios, que el email tenga formato válido y que DNI, email y teléfono sean únicos.

```
--- CLIENT MANAGEMENT ---
1. Create new client
2. Search client
3. List clients
4. Back
Select an option: 1
```

```
Name: Maria Victoria
Surname: Huesca Peláez
DNI/NIF: 12345678A
Email: email1@gmail.com
Phone: 111111111

Client created successfully.
ID client: 1001
```

Buscar cliente

Permite buscar por DNI o por ID interno.

```
--- CLIENT MANAGEMENT ---
1. Create new client
2. Search client
3. List clients
4. Back
Select an option: 2

Search by:
1. DNI
2. ID
3. Go back
Select an option: 1
Introduce DNI/NIF: 12345678A

Client found:
ID:       1001
Name:     María Victoria Huesca Peláez
DNI:      12345678A
Email:    email1@gmail.com
Phone:    111111111
```

```
--- CLIENT MANAGEMENT ---
1. Create new client
2. Search client
3. List clients
4. Back
Select an option: 2

Search by:
1. DNI
2. ID
3. Go back
Select an option: 2
Introduce ID: 1001

Client found:
ID:       1001
Name:     María Victoria Huesca Peláez
DNI:      12345678A
Email:    email1@gmail.com
Phone:    111111111
```

Listar clientes

Muestra todos los clientes registrados en formato tabular.

```
--- CLIENT MANAGEMENT ---
1. Create new client
2. Search client
3. List clients
4. Back
Select an option: 3

ID     | Name                        | DNI          | Email                     | Phone
-------------------------------------------------------------------------------------------
1001   | María Victoria Huesca Peláez | 12345678A    | email1@gmail.com          | 111111111
1002   | Sergio Guzmán               | 12345678B    | email2@gmail.com          | 222222222
```

2.3 Gestión de cuentas

Crear cuenta

Se solicita el ID del cliente titular. El sistema genera automáticamente el número de cuenta en formato IBAN simplificado (ES91210000 + 12 dígitos secuenciales), asigna saldo inicial 0,00 € y registra la fecha de creación.

```
--- ACCOUNT MANAGEMENT ---
1. Create account
2. List client accounts
3. See account information
4. Go back
Select an option: 1

Enter the ID of the account-holding customer: 1001

Account created successfully.
Account number: ES91210000000000000001
Account holder: María Victoria Huesca Peláez (ID: 1001)
Starting balance:    0,00 €
```

Listar cuentas de un cliente

```
--- ACCOUNT MANAGEMENT ---
1. Create account
2. List client accounts
3. See account information
4. Go back
Select an option: 2

Introduce client ID: 1001

Accounts of María Victoria Huesca Peláez:
Account number             | Balance
----------------------------------------
ES91210000000000000001     | 0,00 €
ES91210000000000000002     | 0,00 €
```

Ver información de cuentas

```
--- ACCOUNT MANAGEMENT ---
1. Create account
2. List client accounts
3. See account information
4. Go back
Select an option: 3

Introduce account number: ES91210000000000000001

Account number: ES91210000000000000001
Account holder: María Victoria Huesca Peláez
Balance:            0,00 €
Creation date: 2026-04-17
```

2.4 Operaciones financieras

Depósito

Incrementa el saldo de la cuenta y registra un movimiento de tipo DEPOSIT.

```
--- FINANCIAL OPERATIONS ---
1. Deposit money
2. Withdrawal
3. Transfer between accounts
4. Go back
Select an option: 1

Account number: ES9121000000000000001
Amount to deposit (€): 1000,50

Deposit made successfully.
Account:      ES9121000000000000001
Amount:     +1.000,50 €
New balance: 1.000,50 €
```

Retirada

Reduce el saldo de la cuenta. Valida que el saldo disponible sea suficiente. Registra un movimiento de tipo WITHDRAWAL.

```
--- FINANCIAL OPERATIONS ---
1. Deposit money
2. Withdrawal
3. Transfer between accounts
4. Go back
Select an option: 2

Account number: ES9121000000000000001
Withdrawal amount (€): 200.25

Withdrawal successfully made.
Account:      ES9121000000000000001
Amount:     -200,25 €
New balance: 800,25 €
```

Transferencia

Mueve un importe de una cuenta origen a una cuenta destino. Valida que ambas cuentas existan, que no sean la misma y que el saldo sea suficiente. Registra un movimiento OUTGOING_TRANSFER en la cuenta origen y INCOMING_TRANSFER en la cuenta destino.

```
--- FINANCIAL OPERATIONS ---
1. Deposit money
2. Withdrawal
3. Transfer between accounts
4. Go back
Select an option: 3

Origin account number: ES91210000000000000001
Destination account number: ES91210000000000000003
Amount to transfer (€): 100,25

Transfer successfully completed.
From account:  ES91210000000000000001 → -100,25 €
To account: ES91210000000000000003 → +100,25 €
```

2.5 Consultas

Consultar saldo

```
--- INQUIRIES ---
1. Check balance
2. Movements history
3. Movements by date range
4. Go back
Select an option: 1

Introduce account number: ES91210000000000000001
Actual balance: 700,00 €
```

Historial de movimientos

```
--- INQUIRIES ---
1. Check balance
2. Movements history
3. Movements by date range
4. Go back
Select an option: 2

Introduce account number: ES91210000000000000001

Movements history - ES91210000000000000001:
Date                  | Type                      | Amount
-------------------------------------------------------------
2026-04-17 09:37:23   | OUTGOING_TRANSFER         | -100,25 €
2026-04-17 09:35:45   | WITHDRAWAL                | -200,25 €
2026-04-17 09:35:05   | DEPOSIT                   | +1.000,50 €
```

Movimientos por rango de fechas

Muestra todos los movimientos de una cuenta ordenados del más reciente al más antiguo.

```
--- INQUIRIES ---
1. Check balance
2. Movements history
3. Movements by date range
4. Go back
Select an option: 3

Account number: ES9121000000000000001
Start date (yyyy-MM-dd): 2026-04-17
End date   (yyyy-MM-dd): 2026-04-17

Movements from 2026-04-17 to 2026-04-17:
Date                  | Type                      | Amount
-----------------------------------------------------------------
2026-04-17 09:37:23   | OUTGOING_TRANSFER         | -100,25 €
2026-04-17 09:35:45   | WITHDRAWAL                | -200,25 €
2026-04-17 09:35:05   | DEPOSIT                   | +1.000,50 €
```

Ejemplo de validación de error

```
===================================
 NOVABANK - SYSTEM OPERATIONS
===================================
1. Client management
2. Account management
3. Financial operations
4. Inquiries
5. Exit
Select an option: 2

--- ACCOUNT MANAGEMENT ---
1. Create account
2. List client accounts
3. See account information
4. Go back
Select an option: 3

Introduce account number: 1
ERROR: Could not find an account with number 1.
```

3. Modelo de datos

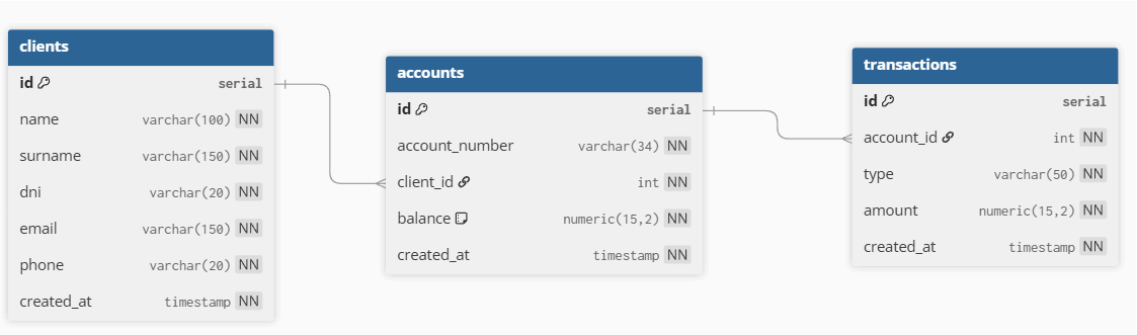
Aunque en este módulo la aplicación no utiliza base de datos, el modelo de datos completo ha sido diseñado desde el principio para su uso en el Módulo 2.

El sistema trabaja con tres entidades principales: Client, Account y Transaction.

Relaciones:

- Un Client puede tener N Accounts (relación 1:N).
- Una Account puede tener N Transactions (relación 1:N).
- Una Transaction pertenece a exactamente una Account.

Diagrama entidad-relación:



4. Decisiones de implementación

Almacenamiento en memoria

En este módulo no se utiliza base de datos. Los datos se almacenan en memoria mediante HashMap, una estructura de la API estándar de Java que permite búsquedas en tiempo constante O(1) usando el identificador como clave.

Cada repositorio gestiona su propio almacén:

- **ClientRepository**:
    - *Clave*: Long (ID del cliente)
    - *Valor*: Client
- **AccountRepository**:
    - *Clave*: String (número de cuenta)
    - *Valor*: Account
- **TransactionRepository:**
    - *Clave*: Long (ID del movimiento)

- *Valor*: Transaction

Los IDs se generan mediante contadores autoincrementales iniciados en 1000 para clientes, 1 para cuentas y 1 para movimientos, simulando el comportamiento de una columna SERIAL en PostgreSQL.

Arquitectura en capas

El proyecto sigue una arquitectura en capas con responsabilidades claramente separadas:

- **model**: entidades del dominio. La clase Account expone métodos credit() y debit() en lugar de un setter de saldo, garantizando que el balance nunca se modifique directamente desde fuera de la entidad.
- **repository**: acceso a datos. Cada repositorio encapsula el HashMap y expone únicamente las operaciones necesarias.
- **service**: lógica de negocio. Centraliza todas las validaciones y orquesta las operaciones entre repositorios.
- **console**: interacción con el usuario. Los menús solo leen entradas y muestran resultados; no contienen lógica de negocio.

Normalización de datos

Para garantizar consistencia en las búsquedas:

- El DNI se normaliza a mayúsculas (toUpperCase()) antes de guardarse.
- El email se normaliza a minúsculas (toLowerCase()) antes de guardarse.

De esta forma los repositorios pueden usar .equals() directamente sin necesidad de comparaciones insensibles a mayúsculas.

Transferencias y consistencia

La operación de transferencia es la más compleja del sistema ya que involucra dos cuentas. El orden de ejecución garantiza la consistencia:

1. Verificar que ambas cuentas existen y que el saldo es suficiente.
2. Retirar el importe de la cuenta origen.
3. Depositar el importe en la cuenta destino.
4. Registrar movimiento OUTGOING_TRANSFER en la cuenta origen.
5. Registrar movimiento INCOMING_TRANSFER en la cuenta destino.

Si se produce cualquier error en los pasos 3-5, el importe se devuelve a la cuenta origen en el bloque catch, dejando el sistema en un estado consistente.

5. Scripts SQL

El archivo src/main/resources/schema.sql contiene los scripts de creación de tablas preparados para el Módulo 2. Es un ejercicio de diseño, no de ejecución, en este módulo la aplicación no utiliza base de datos.

```sql
CREATE TABLE IF NOT EXISTS clients (
   id              SERIAL          PRIMARY KEY,
   name            VARCHAR(100)    NOT NULL,
   surname         VARCHAR(150)    NOT NULL,
   dni             VARCHAR(20)     NOT NULL UNIQUE,
   email           VARCHAR(150)    NOT NULL UNIQUE,
   phone           VARCHAR(20)     NOT NULL UNIQUE,
   created_at      TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS accounts (
   id              SERIAL          PRIMARY KEY,
   account_number VARCHAR(34)      NOT NULL UNIQUE,
   client_id       INT             NOT NULL,
   balance         NUMERIC(15, 2)  NOT NULL DEFAULT 0.00 CHECK (balance >= 0),
   created_at      TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT fk_accounts_client FOREIGN KEY (client_id) REFERENCES clients
(id)
);

CREATE TABLE IF NOT EXISTS transactions (
   id              SERIAL          PRIMARY KEY,
   account_id      INT             NOT NULL,
   type            VARCHAR(50)     NOT NULL CHECK (type IN ('DEPOSIT',
'WITHDRAWAL', 'OUTGOING_TRANSFER', 'INCOMING_TRANSFER')),
   amount          NUMERIC(15, 2)  NOT NULL CHECK (amount > 0),
   created_at      TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT fk_transactions_account FOREIGN KEY (account_id) REFERENCES
accounts (id)
);
```

Explicación de cada tabla:

- **clients**: Almacena los datos personales de cada cliente. Los campos dni, email y phone tienen restricción UNIQUE para garantizar que no existan duplicados. El campo id es autogenerado por la base de datos mediante SERIAL.
- **accounts**: Almacena las cuentas bancarias. Cada cuenta está asociada a un cliente mediante la clave foránea client_id. El campo balance tiene un CHECK (balance >= 0) para garantizar que el saldo nunca sea negativo. El número de cuenta sigue el formato IBAN simplificado (ES91210000 + 12 dígitos).
- **transactions**: Almacena cada movimiento registrado en el sistema. El campo type tiene un CHECK que restringe los valores al conjunto de tipos válidos del sistema. El campo

amount siempre es positivo, el signo de la operación queda determinado por el tipo de movimiento. La clave foránea account_id referencia la cuenta a la que pertenece el movimiento.

6. Testing

El proyecto incluye pruebas unitarias desarrolladas con JUnit 5 que verifican el correcto funcionamiento de la capa de servicio. Se han implementado tres clases de test, una por cada servicio del sistema.

Las pruebas cubren tanto los casos de éxito como los casos de error, siguiendo el patrón Arrange / Act / Assert.

Clases de test

ClientServiceTest

Verifica la creación, búsqueda y listado de clientes. Cubre validaciones de campos obligatorios, formato de email, unicidad de DNI, email y teléfono, y normalización de datos. Los casos probados incluyen: crear cliente con datos válidos, crear cliente con DNI o email duplicado, crear cliente con email de formato inválido, buscar por ID o DNI existente e inexistente, y listar clientes cuando no hay registros.

AccountServiceTest

Verifica la creación de cuentas y la consulta de cuentas por cliente. Cubre los casos de crear una cuenta para un cliente existente e inexistente, buscar una cuenta por número inexistente, listar cuentas de un cliente con y sin cuentas registradas.

TransactionServiceTest

Verifica todas las operaciones financieras. Cubre depósito con importe válido, nulo y negativo; retirada con saldo suficiente e insuficiente; transferencia entre cuentas válidas, a la misma cuenta y con saldo insuficiente; consulta de saldo; historial completo de movimientos; e historial filtrado por rango de fechas.

Ejecución de los tests

```
[INFO] ------------------------------------------------------------------------
PS C:\Users\mvict\fork\NovaBank-Digital-Services> mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------< com.novabank:novabank-digital-services >---------------
[INFO] Building novabank-digital-services 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ novabank-digital-services ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ novabank-digital-services ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ novabank-digital-services ---
[INFO] skip non existing resourceDirectory C:\Users\mvict\fork\NovaBank-Digital-Services\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ novabank-digital-services ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ novabank-digital-services ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.novabank.service.AccountServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.056 s -- in com.novabank.service.AccountServiceTest
[INFO] Running com.novabank.service.ClientServiceTest
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.020 s -- in com.novabank.service.ClientServiceTest
[INFO] Running com.novabank.service.TransactionServiceTest
[INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in com.novabank.service.TransactionServiceTest
```

```
[INFO] Results:
[INFO]
[INFO] Tests run: 63, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.484 s
[INFO] Finished at: 2026-04-17T02:21:21+02:00
[INFO] ------------------------------------------------------------------------
```

7. Control de versiones con Git

El código fuente está alojado en un repositorio público en GitHub. Durante el desarrollo se ha mantenido un flujo de trabajo ordenado con ramas separadas por funcionalidad, siguiendo la estructura recomendada en el enunciado.

Convención de commits

Todos los commits siguen el estándar Conventional Commits, usando los prefijos feat, fix, test y docs para indicar el tipo de cambio realizado en cada unidad de trabajo.

Historial de commits

main	**doc: Readme files and database script for next module**	**victoriahuesca**	**7e05744**	**17 abr. 2026 01:29**
develop	**ref**: minor code refactoring	victoriahuesca	fe74fba	17 abr. 2026 00:57
refactor: address full project code review findings	victoriahuesca	a2c0e31	17 abr. 2026 00:56
feature/clients	**fix**: correct ClientMenu bugs and add target/ to .gitignore	victoriahuesca	9447a90	17 abr. 2026 00:30
Merge branch 'feature/operations' of https://github.com/MVictoriaHuesca/NovaBank-Digit...	victoriahuesca	a671e44	17 abr. 2026 00:21
feature/operations	**refactor**: minor fixes for code quality	victoriahuesca	6977146	17 abr. 2026 00:21
fix: update TransactionServiceTest to use renamed getAccountNumber()	victoriahuesca	147ea4d	17 abr. 2026 00:14
Merge branch 'feature/accounts' into develop	victoriahuesca	7842260	17 abr. 2026 00:06
Merge branch 'feature/operations' of https://github.com/MVictoriaHuesca/NovaBank-...	victoriahuesca	53e5e32	17 abr. 2026 00:02
refactor: minimal changes for code consistency	victoriahuesca	f6eed69	17 abr. 2026 00:02
stash@{0}	WIP on feature/accounts: 7f73bb1 refactor: minimal changes to impro...	victoriahuesca	71137b9	17 abr. 2026 00:00
feature/accounts	**refactor**: minimal changes to improve code consistency	victoriahuesca	7f73bb1	16 abr. 2026 23:57
origin/feature/accounts	**fix**: bad merging, fixed manually	victoriahuesca	fc5ba2c	16 abr. 2026 23:53
Merge branch 'feature/clients' of https://github.com/MVictoriaHuesca/NovaBank-Digital...	victoriahuesca	8d4cc9b	16 abr. 2026 23:49
fix: normalize email to lowercase and improve ClientRepository consistency	victoriahuesca	43b23fc	16 abr. 2026 23:48
fix: normalize email to lowercase and improve ClientRepository consistency	victoriahuesca	95d7831	16 abr. 2026 23:46
fix: Account.debit() updates the balance	victoriahuesca	817881a	16 abr. 2026 23:34
Merge branch 'feature/operations' of https://github.com/MVictoriaHuesca/NovaBank-Di...	victoriahuesca	e923cf2	16 abr. 2026 23:21
refactor: clean up model classes ↩	victoriahuesca	785ab39	16 abr. 2026 23:18
ref: rename numberAccount to accountNumber in all classes	victoriahuesca	6399c41	16 abr. 2026 17:14
test: add TransactionServiceTest	victoriahuesca	3f4873f	16 abr. 2026 17:05
fix: resolve review issues and expand service test coverage	victoriahuesca	e58e649	16 abr. 2026 16:41
feat: AccountServiceTest implementation	victoriahuesca	6f7fd59	16 abr. 2026 14:35
test: unit tests for ClienteService	victoriahuesca	65ea362	16 abr. 2026 13:12
feat: queries sub-menu and transaction history	victoriahuesca	fdd1fb4	16 abr. 2026 12:15
feat: implement TransactionMenu ↩	victoriahuesca	3f73e01	16 abr. 2026 11:40
Merge branch 'develop' of https://github.com/MVictoriaHuesca/NovaBank-Digital-Services...	victoriahuesca	3e539ed	16 abr. 2026 11:17
feat: AccountMenu implementation	victoriahuesca	9f7f920	16 abr. 2026 11:17
Merge branch 'feature/clients' of https://github.com/MVictoriaHuesca/NovaBank-Digital...	victoriahuesca	bdeddc0	16 abr. 2026 10:36
Merge branch 'feature/clients' of https://github.com/MVictoriaHuesca/NovaBank-Digit...	victoriahuesca	54b7583	16 abr. 2026 10:34
feat: implement client menu	victoriahuesca	7476d49	16 abr. 2026 10:30
Merge branch 'feature/clients' of https://github.com/MVictoriaHuesca/NovaBank-Digit...	victoriahuesca	7306ba3	16 abr. 2026 10:28
chore: merged	victoriahuesca	7b72b93	16 abr. 2026 10:27
Merge branch 'feature/operations' of https://github.com/MVictoriaHuesca/NovaBan...	victoriahuesca	ffc55dd	16 abr. 2026 00:54
feat: implement OperationService with deposit, withdrawal and transfer ↩	victoriahuesca	d68d267	16 abr. 2026 00:45
feat: implement OperationService with deposit, withdrawal and transfer	victoriahuesca	bab99ba	16 abr. 2026 00:45
feat: implement AccountService with account creation and lookup	victoriahuesca	ba7ef97	15 abr. 2026 21:21
feat: implement ClientService ↩	victoriahuesca	305b956	15 abr. 2026 20:54
feat: in-memory repository layer and domain model refinements ↩	victoriahuesca	815da28	15 abr. 2026 19:54
feat: complete domain model with Transaction and TransactionType	victoriahuesca	f515470	15 abr. 2026 13:39
feat: partial implementation of the domain model ↩	victoriahuesca	ee2ced5	14 abr. 2026 18:41
build: add project dependencies to pom.xml	victoriahuesca	3694241	14 abr. 2026 12:01
feature/inquiries	**fix**: unnecessary files deleted	victoriahuesca	5499555	14 abr. 2026 11:44
chore: init	victoriahuesca	d255fab	14 abr. 2026 11:24

Estructura de ramas

```
PS C:\Users\mvict\fork\NovaBank-Digital-Services> git branch -a
  develop
  feature/accounts
  feature/clients
  feature/inquiries
  feature/operations
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/develop
  remotes/origin/feature/accounts
  remotes/origin/feature/clients
  remotes/origin/feature/inquiries
  remotes/origin/feature/operations
  remotes/origin/main
```

8. Repositorio en GitHub

El código fuente del proyecto está disponible públicamente en el siguiente repositorio:

https://github.com/MVictoriaHuesca/NovaBank-Digital-Services